FPGA LABORATORY – EXPERIMENT PROBLEM SET

Experiment–1: Full Adder Using Half Adders

Title:
Design and Implementation of a 1-bit Full Adder Using Two Half Adders

Objective:
To design a 1-bit full adder using two half adders and one OR gate, verify its logic, and implement it either on a Boolean FPGA board or on the Basys-3 FPGA board.

Given:
Inputs: A, B, Cin (1-bit each)
Outputs: Sum, Cout (1-bit each)

Tasks:
1. Draw the block diagram using two half adders and an OR gate.
2. Write the truth table of the full adder.
4. Implement the design using Verilog HDL based structural modelling.
5. Simulate the design and verify functionality.
6. Implement on Basys-3/Boolean FPGA board FPGA and verify using switches and LEDs.

Experiment–2: 4-bit Multi-Mode Logic Circuit

Title:
Design and Implementation of a 4-bit Multi-Mode Arithmetic and Logic Circuit Using FPGA

Objective:
To design a logic circuit that performs addition, multiplication, and scaling (A/4) on two 4-bit inputs based on a 2-bit mode signal, and implement it on an FPGA board.

Given:
Inputs:
A[3:0], B[3:0] – 4-bit inputs
Mode[1:0] – 2-bit control signal

Output:
Y[7:0] – 8-bit output

Operation Specification:
Mode = 00 → Y = A + B
Mode = 01 → Y = A × B
Mode = 10 → Y = A/4
Mode = 11 → Invalid mode, Y=> all LED should turn on

Tasks:
1. Draw a block diagram level architecture showing adder, multiplier, shift blocks and selection logic.
2. Implement the design using Verilog HDL (behavioural modelling)
3. Simulate for multiple input combinations covering all modes.
4. Implement on Basys-3/Boolean FPGA board and verify using hardware.

Experiment–3: 3-Input Majority Gate

Title:

Design and Implementation of a 3-Input Majority Gate

Objective:

To design a 3-input majority gate, verify its Boolean logic, and implement it either on a Boolean FPGA board (Spartan-7) or on the Basys-3 FPGA board using switches and LEDs.

Given:

Inputs:
A (1-bit)
B (1-bit)
C (1-bit)

Output:
Y (1-bit)

Design Requirement:

The majority gate must generate logic HIGH (1) at the output when two or more inputs are HIGH.

The design must be realized using basic logic gates (AND, OR) or equivalent combinational logic.

Tasks:

1. Draw the block diagram of a 3-input majority gate using AND and OR gates.
2. Write the truth table of the majority gate.
3. Derive the Boolean expression for the output Y.
4. Implement the design using Verilog HDL with basic logic gates (AND, OR) or logical operators
5. Simulate the design and verify correct majority behaviour for all input combinations.
6. Implement the design on the Basys-3 / Boolean FPGA board (Spartan-7) and verify functionality using switches and LEDs.

Suggested FPGA Mapping :

A → SW0
B → SW1
C → SW2
Y → LED0

Expected Outcome:
The output LED should turn ON when any two or all three input switches are ON, and remain OFF otherwise.

Experiment 4: Loadable Up/Down Counter

Design and implement a **synchronous loadable up/down counter** using **Verilog HDL** and verify its functionality on an **FPGA development board (Basys3/Boolean)**.

Specifications

1. The counter shall be **N-bit wide** (assume **4-bit or 8-bit**, as specified).
2. The counter shall operate using a **single system clock**.
3. A **parallel load input** shall be provided to load an external binary value into the counter.
4. A **mode control input m (2-bit)** shall determine the operation of the counter as follows:
    - m = 2'b10 → Counter increments (UP count)
    - m = 2'b01 → Counter decrements (DOWN count)
    - m = 2'b00 → No operation
    - m = 2'b11 → Counter holds its current value (FREEZE)
5. The counter shall increment or decrement at a **1-second interval**, derived from the system clock.
6. A **reset input** shall asynchronously reset the counter to zero.
7. The counter output shall be displayed using **LEDs** on the FPGA board.

Tasks

- Develop the Verilog HDL code for the counter.
- Generate a **1 Hz enable signal** from the system clock without creating additional clock domains.
- Write appropriate **constraints (XDC)** to map inputs and outputs to FPGA switches, buttons, and LEDs.
- Synthesize, implement, and verify the design on hardware.

Expected Outcome

The FPGA-based counter should correctly perform **up-counting, down-counting, parallel loading, and holding** operations based on the selected mode and control inputs.

Experiment 5: FPGA Implementation of GCD Using Euclid's Algorithm

Design and implement a digital system on an FPGA to compute the **Greatest Common Divisor (GCD)** of two unsigned integers using **Euclid's subtraction-based algorithm**.

The system shall accept two input numbers **A** and **B** through slide switches and start the computation when a **GO** push button is pressed. If either input value is **zero**, the GCD is undefined; in such a case, the system must generate an **error signal** and provide a visual indication by **blinking an LED at 1 Hz frequency**.

For valid inputs, the algorithm shall repeatedly subtract the smaller number from the larger one until both values become equal. When the computation terminates, the final GCD value shall be displayed on LEDs and a **DONE** signal shall be asserted. The system shall remain in its final state until a **RESET** push button is pressed.

Functional Requirements

1. Accept two unsigned integers **A** and **B** using slide switches.
2. Begin computation when the **GO** push button is pressed.
3. Detect invalid inputs (A = 0 or B = 0) and:
    - Assert an **ERROR** signal
    - Blink an LED at **1 Hz**
    - Stop computation
4. For valid inputs, execute Euclid's subtraction algorithm:
    - If A > B, compute A = A − B
    - Else compute B = B − A
    - Repeat until A = B
5. After termination:
    - Output the GCD value (GCD = A) on LEDs
    - Assert the **DONE** signal
6. Maintain output values until **RESET** is activated.

Hardware Mapping Constraints

- **Input A, Input B**: Slide switches
- **GO signal**: Upper push button
- **RESET**: Centre push button
- **GCD Output**: 8 LEDs
- **DONE signal**: Extreme left LED
- **ERROR signal**: Extreme left-2 LED (blinking at 1 Hz)

Design Guidelines

- Implement the control logic using a **Finite State Machine (FSM)**.
- Design a datapath using registers, comparator, and subtractor.
- Generate a **1 Hz clock enable** from the system clock for LED blinking.
- Ensure synthesizable RTL coding practices.

Expected Outcome

Upon successful implementation, the FPGA shall correctly compute and display the GCD of two input numbers, indicate completion using the DONE LED, and visually signal invalid input conditions using a blinking ERROR LED.

Experiment 6: Password-Protected Digital Calculator System Using FPGA

Design and implement a **digital calculator system on an FPGA** using **Verilog HDL** that remains **locked** until a specific sequence of button presses is applied. The calculator performs **addition** and **multiplication** operations once unlocked and provides appropriate visual indications for valid, invalid, and error conditions.

System Requirements

1. **Lock Mechanism**
   - The system has three input push buttons: **S1, S2, and S3**.
   - The calculator unlocks **only if** the following sequence is applied:
   - `S3 → S1 → S2`
   - Any incorrect sequence or pressing of a wrong switch at any stage shall:
     - Activate an **ERROR state**
     - Illuminate an **error LED**
     - Disable all calculator operations
   - The system must be **reset** to retry after an error.
2. **Calculator Activation**
   - Once unlocked:
     - An **ACTIVE LED** must turn ON to indicate that the calculator is enabled.
     - The calculator remains active until reset.
3. **Arithmetic Operations**
   - The calculator accepts:
     - Two unsigned inputs: `a` and `b`
     - A 2-bit mode control `m`
   - Supported operations:

Mode (`m`)	**Operation**
`01`	Addition (`a + b`)
`10`	Multiplication (`a × b`)

4. **Invalid Operation Handling**
   - If the mode input is:
     - `m = 00` or `m = 11`
   - Then:
     - The operation is considered **invalid**
     - An **INVALID OPERATION LED** must **blink continuously at 2 Hz frequency.**

        - No arithmetic result should be produced
5. **Reset Behavior**
    - A asynchronous reset must:
        - Lock the calculator
        - Clear all LEDs
        - Clear error and invalid operation flags

Design Constraints

- Button inputs must be synchronized to the system clock.
- Lock logic must be implemented using a **Finite State Machine (FSM)**.
- Arithmetic logic must be enabled **only when the calculator is unlocked**.
- Invalid operation indication must be implemented using a **clock-based blinking mechanism**.
- The design must be synthesizable and suitable for FPGA implementation.

Expected Outputs

- `active_led` → ON when calculator is unlocked
- `error_led` → ON for incorrect unlock sequence
- `invalid_led` → Blinks for invalid operation mode
- `y` → Result of arithmetic operation

Experiment 7: 10 bit counter

Experiment 8: MAC design

Experiment 9:

Design of a ROM-Based Sequential Accumulator System

Objective:

Design and implement a digital system on the Boolean Board / Basys3 FPGA board that reads a set of preloaded hexadecimal data from a Read-Only Memory (ROM), sequentially accumulates the values, and displays the final result upon pressing a GO signal.

Problem Description

Design a hardware system that performs sequential addition of ten preloaded 8-bit hexadecimal numbers stored in ROM. The summation must begin only when a GO button is pressed and must complete automatically after all ten values have been processed.

Functional Requirements

1. ROM Design

- Implement a 10-location ROM storing 8-bit hexadecimal values.
- Initialize the ROM using a .mem file and $readmemh.
- ROM must use synchronous read operation.
- Address range: 0–9.

2. Control Operation

- Inputs: clk (1 Hz), rst (active-high reset), go (start signal).
- On assertion of go, the system must begin sequential reading of ROM.
- Exactly 10 read operations must occur.
- A counter must track the number of data fetch operations.

3. Accumulation Process

- Each fetched value must be added to an internal accumulator.
- The accumulator must have sufficient width to prevent overflow.
- The summation must complete after exactly 10 additions.

4. Output Requirements

- y: Final accumulated sum.
- busy: High while summation is in progress.
- done: High after completion of 10 additions.
- The result must remain stable until reset.

5. Reset Behavior

- Reset clears address counter, accumulator, busy, and done signals.
- System returns to idle state.

Design Constraints
- Use Verilog HDL only.
- Separate design into Datapath, Controller (FSM), and ROM modules.
- ROM must use synchronous read behavior.
- System clock frequency is 1 Hz.

Expected Operation Sequence
1. System remains idle.
2. User presses go.
3. busy = 1.
4. ROM addresses 0 to 9 are read sequentially.
5. Each value is added to accumulator.
6. After 10th addition: done = 1, busy = 0, final sum appears on y